

# Algorithms 2

## Approximating the Maximum Cut Problem

Daniel Rosenberg & Michael Trushkin

Ariel University

# 1. The Max Cut Problem

---

**Problem 1 Max Cut**

**Given:** An undirected graph  $G = (V, E)$ .

**Find:** A bipartition  $(S, V \setminus S)$  maximising  $e_G(S, V \setminus S)$  — the number of edges crossing the cut.

**Problem 2 Max Cut**

**Given:** An undirected graph  $G = (V, E)$ .

**Find:** A bipartition  $(S, V \setminus S)$  maximising  $e_G(S, V \setminus S)$  — the number of edges crossing the cut.

**Theorem 2** For every graph  $G$ :  $\text{max-cut}(G) \geq \frac{e(G)}{2}$ .

**Problem 3 Max Cut**

**Given:** An undirected graph  $G = (V, E)$ .

**Find:** A bipartition  $(S, V \setminus S)$  maximising  $e_G(S, V \setminus S)$  — the number of edges crossing the cut.

**Theorem 3** For every graph  $G$ :  $\text{max-cut}(G) \geq \frac{e(G)}{2}$ .

**Observation 3** Equivalently: every graph  $G$  has a **spanning bipartite subgraph** with  $\geq \frac{e(G)}{2}$  edges.

## 1.2 Proof of the Theorem

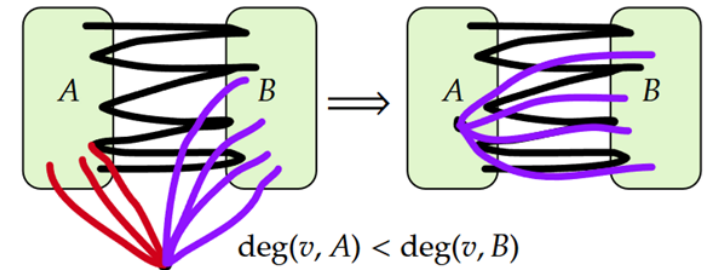
**Proof:** By induction on  $v(G)$ . Base cases  $v(G) = 1, 2$  are trivial.

**Inductive step:** Let  $v \in V(G)$  be arbitrary.

- Apply the IH to  $G - v$ :
  - obtain a bipartition  $(S, V(G) \setminus (S \cup \{v\}))$  with  $\geq \frac{e(G-v)}{2}$  crossing edges.
- Place  $v$  on the side where it has the **fewest** neighbours in  $G$
- Then  $v$  contributes at least  $\deg_G(v)/2$  crossing edges.

Size of resulting cut in  $G$ :

$$\geq \frac{e(G-v)}{2} + \deg_G(v)/2 = \frac{e(G)}{2}. \quad \square$$



## 1.2 Proof of the Theorem

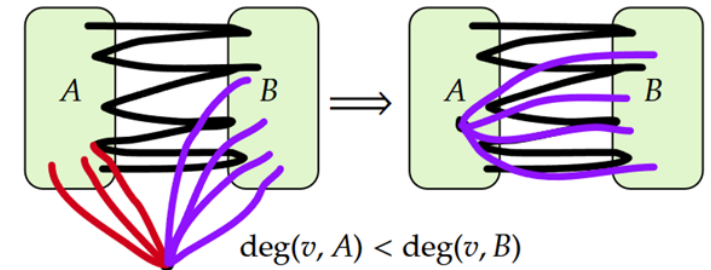
**Proof:** By induction on  $v(G)$ . Base cases  $v(G) = 1, 2$  are trivial.

**Inductive step:** Let  $v \in V(G)$  be arbitrary.

- Apply the IH to  $G - v$ :
  - obtain a bipartition  $(S, V(G) \setminus (S \cup \{v\}))$  with  $\geq \frac{e(G-v)}{2}$  crossing edges.
- Place  $v$  on the side where it has the **fewest** neighbours in  $G$
- Then  $v$  contributes at least  $\deg_G(v)/2$  crossing edges.

Size of resulting cut in  $G$ :

$$\geq \frac{e(G-v)}{2} + \deg_G(v)/2 = \frac{e(G)}{2}. \quad \square$$



**Theorem 5** The above proof yields a poly-time algorithm forming a  $\frac{1}{2}$ -approximation for Max Cut.

## 2. A Randomised Construction

---



---

### Algorithm 1: Rand-Cut

---

```
1:  $A, B \leftarrow \emptyset$ 
2: for each  $v \in V(G)$  do
3:   Toss a fair coin independently
4:   Heads: place  $v$  in  $A$ .   Tails: place  $v$  in  $B$ .
5: end
6: return  $(A, B)$ 
```

---

---

## Algorithm 2: Rand-Cut

---

```

1:  $A, B \leftarrow \emptyset$ 
2: for each  $v \in V(G)$  do
3:   Toss a fair coin independently
4:   Heads: place  $v$  in  $A$ .   Tails: place  $v$  in  $B$ .
5: end
6: return  $(A, B)$ 

```

---

**Observation 5** The randomised algorithm achieves an **expected** cut of size  $\frac{e(G)}{2}$ .

For each edge  $e = uv \in E(G)$ , define the indicator:

$$X_e := \begin{cases} 1 & e \in E_G(A, B) \\ 0 & e \notin E_G(A, B) \end{cases}$$

For each edge  $e = uv \in E(G)$ , define the indicator:

$$X_e := \begin{cases} 1 & e \in E_G(A, B) \\ 0 & e \notin E_G(A, B) \end{cases}$$

Since the coins are **independent**:

$$\mathbb{E}[X_e] = \mathbb{P}[u \in A \wedge v \in B] + \mathbb{P}[u \in B \wedge v \in A] = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$

We may write  $e_G(A, B) = \sum_{e \in E(G)} X_e$ , so by **linearity of expectation**:

$$\mathbb{E}[e_G(A, B)] = \sum_{e \in E(G)} \mathbb{E}[X_e] = \frac{e(G)}{2}$$

For each edge  $e = uv \in E(G)$ , define the indicator:

$$X_e := \begin{cases} 1 & e \in E_G(A, B) \\ 0 & e \notin E_G(A, B) \end{cases}$$

Since the coins are **independent**:

$$\mathbb{E}[X_e] = \mathbb{P}[u \in A \wedge v \in B] + \mathbb{P}[u \in B \wedge v \in A] = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$

We may write  $e_G(A, B) = \sum_{e \in E(G)} X_e$ , so by **linearity of expectation**:

$$\mathbb{E}[e_G(A, B)] = \sum_{e \in E(G)} \mathbb{E}[X_e] = \frac{e(G)}{2}$$

**But how often do we actually get a cut  $\geq \frac{e(G)}{2}$ ?**

**Theorem 6** The algorithm produces a cut of size  $\geq \frac{e(G)}{2}$  with probability  $\geq \frac{1}{e(G)+1}$ .

(This is bad!)

**proof:**

- Let  $p := \mathbb{P}\left[e_G(A, B) \geq \frac{e(G)}{2}\right]$ .
  - **Goal** bound  $p$  from below.

**Theorem 7** The algorithm produces a cut of size  $\geq \frac{e(G)}{2}$  with probability  $\geq \frac{1}{e(G)+1}$ .

(This is bad!)

**proof:**

- Let  $p := \mathbb{P}\left[e_G(A, B) \geq \frac{e(G)}{2}\right]$ .
  - Goal** bound  $p$  from below.

Write the expectation by splitting on the cut value:

$$\begin{aligned} \frac{e(G)}{2} = \mathbb{E}[e_G(A, B)] &= \sum_{i \leq \frac{e(G)}{2} - 1} i \cdot \mathbb{P}[e_G(A, B) = i] + \sum_{i \geq \frac{e(G)}{2}} i \cdot \mathbb{P}[e_G(A, B) = i] \\ &\leq \left(\frac{e(G)}{2} - 1\right) \cdot \underbrace{\sum_{i \leq \frac{e(G)}{2} - 1} \mathbb{P}[e_G(A, B) = i]}_{1-p} + e(G) \cdot \underbrace{\sum_{i \geq \frac{e(G)}{2}} \mathbb{P}[e_G(A, B) = i]}_p \end{aligned}$$

**Theorem 8** The algorithm produces a cut of size  $\geq \frac{e(G)}{2}$  with probability  $\geq \frac{1}{e(G)+1}$ .

(This is bad!)

**proof:**

- Let  $p := \mathbb{P}\left[e_G(A, B) \geq \frac{e(G)}{2}\right]$ .
  - Goal** bound  $p$  from below.

Write the expectation by splitting on the cut value:

$$\begin{aligned} \frac{e(G)}{2} = \mathbb{E}[e_G(A, B)] &= \sum_{i \leq \frac{e(G)}{2} - 1} i \cdot \mathbb{P}[e_G(A, B) = i] + \sum_{i \geq \frac{e(G)}{2}} i \cdot \mathbb{P}[e_G(A, B) = i] \\ &\leq \left(\frac{e(G)}{2} - 1\right) \cdot \underbrace{\sum_{i \leq \frac{e(G)}{2} - 1} \mathbb{P}[e_G(A, B) = i]}_{1-p} + e(G) \cdot \underbrace{\sum_{i \geq \frac{e(G)}{2}} \mathbb{P}[e_G(A, B) = i]}_p \end{aligned}$$

Rearranging:

$$\frac{e(G)}{2} \leq \left(\frac{e(G)}{2} - 1\right)(1 - p) + e(G) \cdot p$$



**Theorem 9** The algorithm produces a cut of size  $\geq \frac{e(G)}{2}$  with probability  $\geq \frac{1}{e(G)+1}$ .

(This is bad!)

$$\bullet p := \mathbb{P}\left[e_G(A, B) \geq \frac{e(G)}{2}\right] \qquad \bullet \frac{e(G)}{2} \leq \left(\frac{e(G)}{2} - 1\right)(1 - p) + e(G) \cdot p$$

Isolating  $p$ :

$$\begin{aligned} \frac{e(G)}{2} &\leq \frac{e(G)}{2} - 1 - p \cdot \left(\frac{e(G)}{2} - 1\right) + e(G) \cdot p \\ 1 &\leq p \cdot \left(e(G) - \frac{e(G)}{2} + 1\right) = p \cdot \left(\frac{e(G)}{2} + 1\right) \\ \Rightarrow p &\geq \frac{1}{\frac{e(G)}{2} + 1} \geq \frac{1}{e(G) + 1}. \quad \square \end{aligned}$$

### 3. De-randomisation

---

**Goal:** Turn the randomised algorithm into a **deterministic** one with the same guarantee.

**Key tool:** the **method of conditional expectations**.

**Goal:** Turn the randomised algorithm into a **deterministic** one with the same guarantee.

**Key tool:** the **method of conditional expectations**.

### Definition 5 Conditional Expectation

For discrete RVs  $X, Y$  and value  $z$  in the range of  $Y$ :

$$\mathbb{E}[X \mid Y = z] := \sum_x x \cdot \mathbb{P}[X = x \mid Y = z]$$

We write  $\mathbb{E}[X \mid Y]$  for the **function**  $z \mapsto \mathbb{E}[X \mid Y = z]$ .

**Goal:** Turn the randomised algorithm into a **deterministic** one with the same guarantee.

**Key tool:** the **method of conditional expectations**.

### Definition 6 Conditional Expectation

For discrete RVs  $X, Y$  and value  $z$  in the range of  $Y$ :

$$\mathbb{E}[X \mid Y = z] := \sum_x x \cdot \mathbb{P}[X = x \mid Y = z]$$

We write  $\mathbb{E}[X \mid Y]$  for the **function**  $z \mapsto \mathbb{E}[X \mid Y = z]$ .

### Theorem 12 Law of Total Expectation

For discrete RVs  $X, Y$ :

$$\mathbb{E}[X] = \sum_{z \in \text{range}(Y)} \mathbb{P}[Y = z] \cdot \mathbb{E}[X \mid Y = z]$$

Let  $v_1, \dots, v_n$  be an ordering of the vertices. Suppose  $v_1, \dots, v_i$  have already been placed in  $A$  or  $B$ .

By the **Law of Total Expectation**, for vertex  $v_{i+1}$ :

$$\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}] = \frac{1}{2} \cdot \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in A] + \frac{1}{2} \cdot \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in B]$$

Let:

$$\begin{aligned} \bullet \mathbb{E}_A &= \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in A] & \bullet \mathbb{E}_B &= \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in B] \end{aligned}$$

Let  $v_1, \dots, v_n$  be an ordering of the vertices. Suppose  $v_1, \dots, v_i$  have already been placed in  $A$  or  $B$ .

By the **Law of Total Expectation**, for vertex  $v_{i+1}$ :

$$\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}] = \frac{1}{2} \cdot \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in A] + \frac{1}{2} \cdot \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in B]$$

Let:

$$\bullet \mathbb{E}_A = \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in A] \qquad \bullet \mathbb{E}_B = \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in B]$$

So:

$$2\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}] = \mathbb{E}_A + \mathbb{E}_B$$

Let  $v_1, \dots, v_n$  be an ordering of the vertices. Suppose  $v_1, \dots, v_i$  have already been placed in  $A$  or  $B$ .

By the **Law of Total Expectation**, for vertex  $v_{i+1}$ :

$$\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}] = \frac{1}{2} \cdot \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in A] + \frac{1}{2} \cdot \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in B]$$

Let:

$$\bullet \mathbb{E}_A = \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in A] \qquad \bullet \mathbb{E}_B = \mathbb{E}[e_G(A, B) \mid \dots, v_{i+1} \in B]$$

So:

$$2\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}] = \mathbb{E}_A + \mathbb{E}_B$$

**Observation 8** Either  $\mathbb{E}_A \geq \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}]$  or  $\mathbb{E}_B \geq \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}]$ .



## 4. The Deterministic Algorithm

---

**Assuming:**  $\mathbb{E}_A$  and  $\mathbb{E}_B$  can be computed efficiently we have the following algorithm.

---

## Algorithm 3: Derand-Max-Cut

---

```

1:  $v_1, \dots, v_n \leftarrow$  an arbitrary ordering of  $V$ 
2:  $A, B \leftarrow \emptyset$ 
3: for  $i = 1$  to  $n$  do
4:    $E_A \leftarrow$  Compute  $\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_{i-1} \text{ placed} \wedge v_i \in A]$ 
5:    $E_B \leftarrow$  Compute  $\mathbb{E}[e_G(A, B) \mid v_1, \dots, v_{i-1} \text{ placed} \wedge v_i \in B]$ 
6:   If  $E_A \geq E_B$  place  $v_i$  in  $A$ , else place  $v_i$  in  $B$ 
7: end
8: return  $(A, B)$ 

```

---

**Theorem 13** Derand-Max-Cut is a deterministic poly-time  $\frac{1}{2}$ -approximation for Max Cut.

**Proof (By induction):**

$$\text{I.H. } \forall i < k, \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed}] \geq \frac{e(G)}{2}$$

Assume without loss of generality that  $\mathbb{E}_A \geq \mathbb{E}_B$ .

- By assumption we have  $\mathbb{E}_A \geq \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_{k-1} \text{ placed}] \geq \frac{e(G)}{2}$ .

$\implies$  for  $k = n$  we have  $e_G(A, B) = \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_n \text{ placed}] \geq \frac{e(G)}{2}$  as required.

For each step  $i$ , we need to compute efficiently:

$$\mathbb{E}_A := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in A]$$

$$\mathbb{E}_B := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in B]$$

For each step  $i$ , we need to compute efficiently:

$$\mathbb{E}_A := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in A]$$

$$\mathbb{E}_B := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in B]$$

Let  $\tilde{A}, \tilde{B}$  be the **current actual sets** with  $v_1, \dots, v_i$  placed. Define:

$$\ell := e(G) - \deg_G(v_{i+1}, \tilde{A}) - \deg_G(v_{i+1}, \tilde{B}) - e_G(\tilde{A}) - e_G(\tilde{B}) - e_G(\tilde{A}, \tilde{B})$$

$\ell = \#$  edges among the **remaining** vertices  $v_{i+2}, \dots, v_n$  (still random).

For each step  $i$ , we need to compute efficiently:

$$\mathbb{E}_A := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in A]$$

$$\mathbb{E}_B := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in B]$$

Let  $\tilde{A}, \tilde{B}$  be the **current actual sets** with  $v_1, \dots, v_i$  placed. Define:

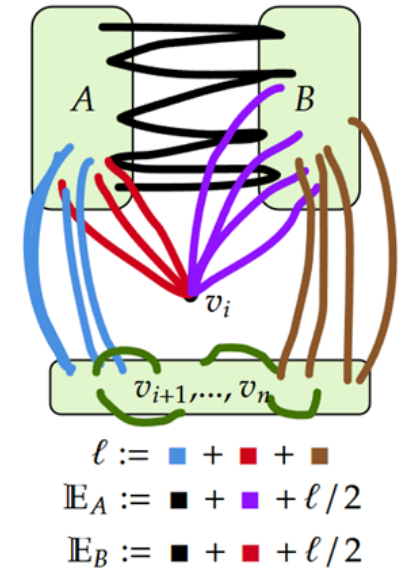
$$\ell := e(G) - \deg_G(v_{i+1}, \tilde{A}) - \deg_G(v_{i+1}, \tilde{B}) - e_G(\tilde{A}) - e_G(\tilde{B}) - e_G(\tilde{A}, \tilde{B})$$

$\ell = \#$  edges among the **remaining** vertices  $v_{i+2}, \dots, v_n$  (still random).

Then:

$$\mathbb{E}_A = e_G(\tilde{A}, \tilde{B}) + \deg_G(v_{i+1}, \tilde{B}) + \frac{\ell}{2}$$

$$\mathbb{E}_B = e_G(\tilde{A}, \tilde{B}) + \deg_G(v_{i+1}, \tilde{A}) + \frac{\ell}{2}$$



For each step  $i$ , we need to compute efficiently:

$$\mathbb{E}_A := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in A]$$

$$\mathbb{E}_B := \mathbb{E}[e_G(A, B) \mid v_1, \dots, v_i \text{ placed} \wedge v_{i+1} \in B]$$

Let  $\tilde{A}, \tilde{B}$  be the **current actual sets** with  $v_1, \dots, v_i$  placed. Define:

$$\ell := e(G) - \deg_G(v_{i+1}, \tilde{A}) - \deg_G(v_{i+1}, \tilde{B}) - e_G(\tilde{A}) - e_G(\tilde{B}) - e_G(\tilde{A}, \tilde{B})$$

$\ell = \#$  edges among the **remaining** vertices  $v_{i+2}, \dots, v_n$  (still random).

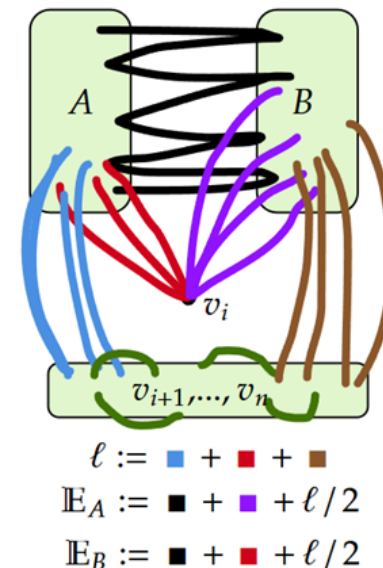
Then:

$$\mathbb{E}_A = e_G(\tilde{A}, \tilde{B}) + \deg_G(v_{i+1}, \tilde{B}) + \frac{\ell}{2}$$

$$\mathbb{E}_B = e_G(\tilde{A}, \tilde{B}) + \deg_G(v_{i+1}, \tilde{A}) + \frac{\ell}{2}$$

Therefore:

$$|\mathbb{E}_A - \mathbb{E}_B| = |\deg_G(v_{i+1}, \tilde{B}) - \deg_G(v_{i+1}, \tilde{A})|$$



$$|\mathbb{E}_A - \mathbb{E}_B| = |\deg_G(v_{i+1}, \tilde{B}) - \deg_G(v_{i+1}, \tilde{A})|$$

Therefore:

$$\mathbb{E}_A \geq \mathbb{E}_B \iff \deg_G(v_{i+1}, \tilde{B}) \geq \deg_G(v_{i+1}, \tilde{A})$$

$$|\mathbb{E}_A - \mathbb{E}_B| = |\deg_G(v_{i+1}, \tilde{B}) - \deg_G(v_{i+1}, \tilde{A})|$$

Therefore:

$$\mathbb{E}_A \geq \mathbb{E}_B \iff \deg_G(v_{i+1}, \tilde{B}) \geq \deg_G(v_{i+1}, \tilde{A})$$

So we may write:

---

**Algorithm 5: Derand-Max-Cut**

---

```
1:  $v_1, \dots, v_n \leftarrow$  an arbitrary ordering of  $V$ 
2:  $A, B \leftarrow \emptyset$ 
3: for  $i = 1$  to  $n$  do
4:   if  $\deg_G(v_{i+1}, B) \geq \deg_G(v_{i+1}, A)$  place  $v_i$  in  $A$ , else place  $v_i$  in  $B$ 
5: end
6: return  $(A, B)$ 
```

---

**This is the first “algorithm” we saw in those slides.**